GraphHunter

Manual del analista

Lucas Sotomayor
Base4Security

13 de mayo de 2026

Índice general

1. Introducción **1**
- 1.1. Qué es GraphHunter . . . 1
- 1.2. Cuándo usarlo . . . 1
- 1.3. Flujo típico de un turno . . . 1
- 1.4. Este manual . . . 2
- 1.5. Convenciones . . . 2

2. Instalación **3**
- 2.1. Requisitos . . . 3
- 2.2. Desde los binarios distribuídos . . . 3
- 2.3. Desde fuente . . . 3
- 2.4. Primer arranque . . . 4
- 2.5. Verificación . . . 4

3. Primeros pasos **5**
- 3.1. Datos demo . . . 5
- 3.2. 1. Crear sesión . . . 5
- 3.3. 2. Ingestar . . . 5
- 3.4. 3. Explorar el catálogo . . . 5
- 3.5. 4. Correr el primer hunt . . . 6
- 3.6. 5. Modo explorador . . . 6
- 3.7. 6. Etiquetar y anotar . . . 6
- 3.8. 7. Exportar . . . 6
- 3.9. Y ahora? . . . 6

4. Ingesta de datos **7**
- 4.1. Formatos soportados . . . 7
- 4.2. One-shot vs streaming . . . 7
- 4.3. Field mapping . . . 7
- 4.4. Ingestor asistido por LLM . . . 8
- 4.5. Archivos grandes . . . 8
- 4.6. Dead Letter Queue . . . 9
- 4.7. Drift . . . 9

5. Hipótesis y DSL **10**
- 5.1. La idea . . . 10
- 5.2. Sintaxis mínima . . . 10
  - 5.2.1. Un solo paso . . . 10
  - 5.2.2. Cadena de pasos . . . 10
  - 5.2.3. Comodín . . . 10
  - 5.2.4. Predicados . . . 10
  - 5.2.5. Ventana temporal . . . 11

5.3. Catálogo built-in . . . 11
5.4. Ejemplos clásicos . . . 11
5.4.1. Lateral movement . . . 11
5.4.2. Exfiltración sospechosa . . . 11
5.4.3. Cadena de spawning Office . . . 11
5.4.4. DNS tunneling . . . 11
5.5. Buenas y malas hipótesis . . . 11
5.6. Validar sin ejecutar . . . 12
5.7. Guardar hipótesis . . . 12

6. Modo caza **13**
6.1. Ejecutar un hunt . . . 13
6.2. Paginación . . . 13
6.3. Scoring: los 5 componentes . . . 13
6.4. Interpretar un path . . . 14
6.5. Workflow de priorización . . . 14
6.6. Explicar un score . . . 14
6.7. Diferencial entre dos hunts . . . 14
6.8. Recomputar scoring . . . 15

7. Modo explorador **16**
7.1. Qué es . . . 16
7.2. Buscar entidades . . . 16
7.3. Panel de detalle . . . 16
7.4. Expandir vecinos . . . 16
7.5. Eventos del nodo . . . 17
7.6. Subgrafo . . . 17
7.7. Cuándo pasar de caza a exploración . . . 17
7.8. Cuándo volver a caza . . . 17

8. Notas y tags **18**
8.1. Tags . . . 18
8.2. Listar por tag . . . 18
8.3. Notas . . . 18
8.4. Workflow de reporte . . . 19
8.5. Interacción con DSL . . . 19

9. Exportación **20**
9.1. IOCs . . . 20
9.2. Publicar IOCs a Sentinel . . . 20
9.3. OCSF . . . 20
9.4. Subgrafos . . . 21
9.5. Resultados de hunt . . . 21
9.6. Sigma . . . 21
9.7. Abrir ticket . . . 21
9.8. Workflow típico de entrega . . . 21

10. Integraciones **22**
10.1. Azure Sentinel . . . 22
10.1.1. Ingesta streaming . . . 22
10.1.2. Publish IOCs . . . 22
10.1.3. Estado . . . 22
10.2. MCP (Model Context Protocol) . . . 22

10.2.1. Setup . . . 22
10.2.2. Qué podés pedir . . . 23
10.2.3. Debug . . . 23
10.3. AI backends (LLM) . . . 23
10.4. GeoIP (MaxMind) . . . 23
10.5. Tickets (Jira / ServiceNow) . . . 24
10.5.1. Jira . . . 24
10.5.2. ServiceNow . . . 24

11. Troubleshooting **25**
11.1. 401 Unauthorized . . . 25
11.2. Connection refused . . . 25
11.3. HTML en lugar de JSON . . . 25
11.4. Out of memory con archivos grandes . . . 25
11.5. MCP client no ve tools nuevas . . . 26
11.6. Datos demo fallan . . . 26
11.7. GNN scoring siempre 0 . . . 26
11.8. Hunt devuelve 0 resultados . . . 26
11.9. DLQ creciendo . . . 26
11.10La app está lenta . . . 27
11.11Dónde reportar bugs . . . 27

12. Preguntas frecuentes **28**
12.1. ¿Reemplaza a mi SIEM? . . . 28
12.2. ¿Cuándo me conviene activar scoring? . . . 28
12.3. ¿Cuándo conviene activar el GNN threat? . . . 28
12.4. ¿Puedo tener varias sesiones abiertas? . . . 28
12.5. ¿El LLM ve mis datos? . . . 28
12.6. ¿Qué pasa si ingesta un log corrupto? . . . 28
12.7. ¿Cómo interpretar un path con gnn_threat = 0? . . . 29
12.8. ¿Qué pasa si cambio el mapping sobre datos ya ingeridos? . . . 29
12.9. ¿Puedo mover una sesión entre máquinas? . . . 29
12.10¿Por qué el Host corporativo aparece con rarity alta? . . . 29
12.11¿Puedo compartir un hunt con un colega? . . . 29
12.12¿El grafo vive en disco? . . . 29
12.13¿Qué diferencia hay entre k, window y min_score? . . . 29
12.14¿Por qué algunos endpoints requieren una flag de entorno? . . . 29
12.15¿Mi hunt tarda mucho. Hay algo que pueda activar? . . . 30
12.16¿Cuándo se activa el compilador IA de *ingest*? . . . 30
12.17¿Por qué mi sesión guardada cambió de formato? . . . 30
12.18¿Por qué mi sesión grande abre rápido pero el primer hunt tarda más? . . . 31
12.19¿Cómo escribo una hipótesis con forma de estrella o árbol? . . . 31
12.20¿Hay forma de correr varias hipótesis a la vez? . . . 31

A. Glosario **33**

B. Atajos de teclado **35**
B.1. Global . . . 35
B.2. Navegación . . . 35
B.3. Hunt . . . 36
B.4. Explorador . . . 36
B.5. Tags y notas . . . 36
B.6. Multi-selección . . . 36

C. Datos demo **37**
C.1. Qué hay en demo_data/ . . . 37
C.2. Incidentes embebidos . . . 37
C.2.1. fortigate_sample . . . 37
C.2.2. sysmon_sample . . . 37
C.2.3. sentinel_signin . . . 37
C.2.4. cognito_sample . . . 38
C.2.5. iis_sample . . . 38
C.3. Datos reales . . . 38
C.4. Reset . . . 38

Capítulo 1

Introducción

1.1. Qué es GraphHunter

GraphHunter es una herramienta de caza de amenazas sobre grafos temporales. Toma logs de seguridad de distintas fuentes (Sysmon, Sentinel, FortiAnalyzer, IIS, Cognito, CSVs genéricos) y construye un grafo donde los nodos son entidades (usuarios, hosts, IPs, procesos, archivos, dominios) y las aristas son relaciones observadas en el tiempo (autenticar, ejecutar, conectar, escribir).

Sobre ese grafo, el analista describe *hipótesis* con un DSL compacto y el sistema devuelve los paths que las satisfacen, scoreados por anomalía.

Véase también el libro *Algoritmos* (`docs/algoritmos/algoritmos.pdf`) si querés entender qué pasa por detrás cuando hacés clic en "Run hunt": cómo el motor elige el operador de matching, qué índices consulta, por qué el primer *hunt* sobre una sesión recién cargada tarda más, y qué nuevas *shapes* de hipótesis acepta el DSL.

1.2. Cuándo usarlo

- **Caso típico**: un incidente (alerta, ticket CSIRT, sospecha de lateral movement) que requiere reconstruir qué pasó y quién participó a partir de logs dispersos.
- **Investigación proactiva**: formular hipótesis de TTPs ATT&CK sobre logs históricos.
- **Análisis post-mortem**: documentar una cadena de eventos y exportar IOCs u OCSF para entregar al cliente.

GraphHunter no reemplaza al SIEM. El SIEM es bueno para búsquedas textuales y alertas por regla; GraphHunter es bueno para preguntas estructurales: "¿qué usuarios que autenticaron anoche también escribieron en rutas sensibles dentro de la siguiente hora?".

1.3. Flujo típico de un turno

1. Abrir la app, crear una sesión nueva.
2. Ingestar los logs relevantes (drag-and-drop, o conectar a Sentinel).
3. Inspeccionar el catálogo: ¿qué tipos de entidad apareció? ¿qué relaciones se observaron?
4. Escribir una hipótesis en DSL y correr el hunt.
5. Revisar los paths devueltos ordenados por score; explorar los de mayor rareza en el *modo explorador*.

6. Etiquetar entidades relevantes, agregar notas y exportar resultados.

El capítulo 3 te guía por este flujo end-to-end con datos demo en unos 10 minutos.

1.4. Este manual

Este libro es para el analista. No requiere saber Rust, TypeScript ni los internals. Si querés entender cómo está construido GraphHunter, ver el libro de Arquitectura. Si necesitás la referencia de API o DSL con detalle de contratos, ver Especificación.

1.5. Convenciones

- `Texto monoespaciado` refiere a comandos, endpoints, código o valores literales.
- `rutas/entre/barras` son rutas del proyecto.
- Los ejemplos asumen que la app corre en `127.0.0.1:37891`. Si configuraste otro puerto, ajustá.

Capítulo 2

Instalación

2.1. Requisitos

- **Sistema**: Windows 10/11, macOS 12+, Linux moderno (glibc 2.31+).
- **Windows**: WebView2 Runtime (preinstalado en Win11; en Win10 se descarga desde `https://developer.mi`
- **Rust toolchain**: stable 1.80+ (solo si compilás desde fuente).
- **Node.js 18+**: para el servidor MCP.

2.2. Desde los binarios distribuídos

El paquete oficial incluye la app Tauri, el CLI y el servidor MCP. Descargá el instalador para tu plataforma y corrélo. El instalador ubica:

- `GraphHunter.exe` (o `.app` / AppImage).
- `graphhunter-cli` en `\$PATH`.
- `graph-hunter-mcp/` como paquete Node junto al app, listo para referenciar desde tu cliente MCP.

2.3. Desde fuente

Si necesitás compilar:

```
# Clonar
git clone https://github.com/base4sec/graphhunter
cd graphhunter

# Desktop app
cd apps/tauri
cargo tauri build

# CLI
cd ../..
cargo build -p graph_hunter_cli --release

# MCP server
cd platform/mcp
npm install && npm run build
```

2.4. Primer arranque

Al abrir GraphHunter por primera vez, la app imprime en su consola de arranque (visible en Ver → Logs del sistema):

```
GraphHunter API listening on http://127.0.0.1:37891
GRAPHHUNTER_API_TOKEN=<uuid>
```

Copiá ese UUID. Lo vas a necesitar para:

- Configurar el servidor MCP (cap. 10).
- Usar el CLI (`graphhunter-cli --token <uuid> ...`).
- Llamar la API desde scripts.

El token se regenera cada arranque salvo que fijes `GRAPHHUNTER_API_TOKEN` en `.env` junto al binario.

2.5. Verificación

```
curl -H "Authorization: Bearer <uuid>" \
     http://127.0.0.1:37891/health
# -> {"status":"ok"}
```

Si devuelve `"ok"`, estás listo para ir al capítulo 3.

Capítulo 3

Primeros pasos

Esta guía te lleva de la app recién instalada a un hunt con resultados y scoring, sobre datos sintéticos, en unos diez minutos.

3.1. Datos demo

El repositorio incluye `demo_data/`. Para esta guía usamos `demo_data/fortigate_sample.csv`: 50k líneas de FortiAnalyzer sintético con un incidente de lateral movement embebido.

Si querés datos reales, ver apéndice C.

3.2. 1. Crear sesión

Abrí la app. *Archivo → Nueva sesión*. Dale nombre (ej. `quickstart-forti`). Aparece el panel de ingesta a la izquierda.

3.3. 2. Ingestar

Arrastrá `demo_data/fortigate_sample.csv` al panel *Ingesta*. El sistema:

1. Detecta el formato (FortiAnalyzer CSV) y carga un `FieldConfig` preconfigurado.
2. Muestra preview con los roles de campo asignados (`TIME`, `ACTOR`, `TARGET`, `ACTION`).
3. Pide confirmar o ajustar el mapping.

Confirmar con los defaults. La ingesta tarda 15-30 segundos para el dataset demo. La barra de progreso muestra eventos procesados.

3.4. 3. Explorar el catálogo

En el panel derecho, pestaña *Catálogo*. Deberías ver:

- `IP` — miles de direcciones observadas.
- `User` — cuentas con actividad.
- `Service` — apps identificadas por FortiOS.

Con conteos. Si algo se ve raro (conteos en 0, tipos inesperados), revisá el field mapping — probablemente un rol quedó mal.

3.5. 4. Correr el primer hunt

Abrir *Hunt* (panel central). En la caja de hipótesis:

```
IP -[Connect]-> IP {k=3}
```

Click *Run*. El sistema devuelve paths ordenados por score. Click en un path para ver su desglose:

- *Entity rarity*: 0.82 (las IPs son raras en conjunto).
- *Edge rarity*: 0.91 (esta secuencia es inusual).
- *Temporal novelty*: 0.64 (las IPs aparecieron tarde en la ventana).

3.6. 5. Modo explorador

Hacer doble click sobre una IP interesante → abre el modo *explorador*. Ves los vecinos del nodo, las relaciones, la timeline. Podés:

- `Expand` para ver vecinos.
- `Expand grouped` si hay muchos, agrupados por tipo de relación.
- Ver *eventos* del nodo en orden temporal.

3.7. 6. Etiquetar y anotar

Sobre un nodo que te parece relevante, click derecho → *Tag* → escribir "sospechoso-c2". Agregar una nota: *Note* → "IP externa con patrón de beacon, ver hunt #2".

3.8. 7. Exportar

Archivo → *Exportar* → *IOCs*. Elegí formato STIX. Guardá el archivo. Ya tenés evidencia lista para entregar.

3.9. Y ahora?

- Entender los formatos: cap. 4.
- Escribir hipótesis más elaboradas: cap. 5.
- Interpretar scoring: cap. 6.
- Integrar con tu SIEM: cap. 10.

Capítulo 4

Ingesta de datos

4.1. Formatos soportados

- **Sysmon**: EVTX exportado o XML de `Microsoft-Windows-Sysmon/Operational`. IDs 1, 3, 7, 8, 10, 11, 13, 22, 23.
- **Sentinel / Log Analytics**: JSON de queries KQL o stream vía conector. Tablas: `SigninLogs`, `AuditLogs`, `SecurityEvent`, `DeviceProcessEvents`, `AzureActivity`, `SecurityAlert`.
- **FortiAnalyzer**: CSV con esquema propietario (`date`, `time`, `srcip`, `dstip`, etc.).
- **AWS Cognito**: JSON de CloudWatch Logs para eventos de Cognito.
- **IIS**: W3C Extended Log con `\#Fields`.
- **CSV genérico**: cualquier CSV con `FieldConfig` mapeado.
- **Generic (texto)**: fallback heurístico para formatos sin parser específico.

Para detalle del mapeo de cada formato a entidades del grafo, ver capítulo *Formatos de log* en el libro de Especificación.

4.2. One-shot vs streaming

One-shot: subís un archivo (drag-and-drop, o *Ingesta → Upload*). El sistema lo procesa íntegro, construye el grafo, cierra. Óptimo para investigaciones sobre logs históricos.

Streaming: el conector de Sentinel abre una query KQL continua; los eventos llegan en batches y se agregan al grafo vivo. Óptimo para monitoreo activo o construcción progresiva de una ventana de análisis.

Elegir en *Ingesta → Modo*.

4.3. Field mapping

Todo formato termina produciendo un `FieldConfig` — un objeto que mapea campos del log a roles semánticos del grafo:

- `TIME` — timestamp (obligatorio).
- `ACTOR` — quién hizo la acción (usuario, proceso).
- `ACTION` — qué acción fue (autenticar, conectar).

- `TARGET` — sobre qué recurrió (host, archivo, IP).
- `INFO` — atributos adicionales (puerto, bytes).

Para formatos conocidos (Sysmon, Sentinel, Forti, Cognito, IIS), el `FieldConfig` viene preconfigurado. Para CSV genérico, la UI ofrece preview y asignación de roles:

1. Arrastrás el archivo.
2. Ves un preview de las primeras 100 filas con los roles propuestos.
3. Podés ajustar cada columna a otro rol con el dropdown.
4. Confirmar → el sistema aplica el mapping e ingesta.

4.4. Ingestor asistido por LLM

Para formatos no reconocidos (y para evolución de formatos conocidos), GraphHunter incluye el carril *agéntico*. El flujo típico:

1. Si el sistema no reconoce el formato, *Ingesta → Asistente → Generar mapping*.
2. El LLM lee una muestra y propone un `FieldConfig`.
3. El preview muestra la propuesta; revisás los roles y mapeo a entidades.
4. Aprobar → el mapping se compila a VRL y queda disponible.
5. Próxima ingesta del mismo formato lo usa automáticamente.

El LLM nunca ingesta eventos en caliente. Propone datos, el compilador determinístico emite código, el runtime lo ejecuta. Ver libro de Arquitectura (cap. *Ingestor agéntico*) y Especificación (cap. *Formatos de log*) para el detalle.

4.5. Archivos grandes

Para archivos de decenas de GB (EVTX de dominio grande, exportación de varios meses de Sentinel):

- **Preferir streaming** al modo one-shot.
- Activar *Ingesta → Opciones → Ingesta por disco* si el archivo supera el RAM disponible. El sistema usa un store intermedio en disco.
- Ajustar `GRAPHHUNTER_INGEST_BATCH_SIZE` a 5000-20000 según memoria.
- Para casos extremos (100 GB+), considerar partir el archivo antes con herramientas externas y ingestar por partes a la misma sesión.

4.6. Dead Letter Queue

Eventos que se parsean pero violan invariantes (timestamp nulo, entidad faltante, relación inválida) van a la DLQ — no se pierden, no corrompen el grafo. Para revisar:

- *Panel → DLQ* muestra todos los eventos pendientes con la razón de rechazo.
- Corregido el mapping (o el evento), *Reingerir* vuelve a intentar.
- *Purgar* vacía la cola sin reintentar.

Una DLQ llena es señal de mapping mal configurado. Antes de purgar, inspeccionar qué eventos fallan y por qué.

4.7. Drift

Cuando un formato que ingerís hace tiempo cambia aguas arriba (un campo nuevo, un separador distinto), el monitor de drift lo detecta y abre una propuesta de *re-review* en el panel agéntico. El sistema sigue ingestando con el mapping actual mientras tanto. Revisar y aprobar el mapping actualizado cuando te venga bien.

Capítulo 5

Hipótesis y DSL

5.1. La idea

Una *hipótesis* es una pregunta sobre el grafo expresada en términos de entidades y relaciones. El DSL describe "formas" que el sistema busca:

```
Entity -[Relation]-> Entity
```

Leída como "una `Entity` conectada por una `Relation` a otra `Entity`". El resultado son todos los paths concretos del grafo que encajan.

5.2. Sintaxis mínima

5.2.1. Un solo paso

```
User -[Auth]-> Host
```

Busca usuarios que se autenticaron en algún host.

5.2.2. Cadena de pasos

```
User -[Auth]-> Host -[Connect]-> Host {k=3}
```

Usuario que autenticó contra un host, y desde ese host se abrió una conexión a otro. `k=3` es una cota de simplicidad: acota la explosión combinatoria limitando longitud o ramificación.

5.2.3. Comodín

```
* -[Execute]-> Process
```

Cualquier entidad que ejecutó un proceso. Útil cuando no sabés de dónde mirar.

5.2.4. Predicados

```
Process {name="cmd.exe"} -[Spawn]-> Process {name="powershell.exe"}
```

Restringe los nodos por propiedad. Comparadores: `=`, `!=`, `>`, `<`, `>=`, `<=`, `like` (strings).

5.2.5. Ventana temporal

```
Host -[DNS]-> Domain {matches_regex(value, "^[a-f0-9]{32}\.")}
{window=1h}
```

Limita la separación máxima entre eventos del path. Unidades: `s`, `m`, `h`, `d`.

5.3. Catálogo built-in

Tipos de entidad: `IP`, `Host`, `User`, `Process`, `File`, `Domain`, `Registry`, `URL`, `Service`, más el comodín `*`.

Relaciones: `Auth`, `Connect`, `Execute`, `Read`, `Write`, `DNS`, `Modify`, `Spawn`, `Delete`, `*`.

Predicados: `in_subnet(ip, cidr)`, `matches_regex(field, pattern)`, `is_weekend()`, `within_hours(sta end)`, `tag(name)`.

Para la gramática formal completa ver el libro de Especificación, cap. *DSL*.

5.4. Ejemplos clásicos

5.4.1. Lateral movement

```
User -[Auth]-> Host -[Connect]-> Host
  {window=30m, k=3}
```

5.4.2. Exfiltración sospechosa

```
Process -[Read]-> File
  -[Connect]-> IP {not in_subnet(value, "10.0.0.0/8")}
  {window=1h}
```

Proceso que lee un archivo y después se conecta a una IP externa dentro de una hora.

5.4.3. Cadena de spawning Office

```
Process {name="winword.exe"}
  -[Spawn]-> Process {name="cmd.exe"}
  -[Spawn]-> Process {name="powershell.exe"}
```

5.4.4. DNS tunneling

```
Host -[DNS]-> Domain
  {matches_regex(value, "^[a-f0-9]{32}\.")}
  {window=1h}
```

5.5. Buenas y malas hipótesis

Buenas:

- Específicas sobre tipos y relaciones (no comodín en todo).
- Con `k` y `window` acotadas.

- Con predicados que descartan casos comunes (ej. excluir subnet corporativa para mirar tráfico externo).

Malas:

- `* -[*]-> *` — devuelve todo el grafo, no informa.
- Sin `k` ni `window` sobre un grafo grande — puede tomar minutos o fallar por presión de memoria.
- Demasiado restrictivas — cero resultados; considerá relajar un predicado y revisar.

5.6. Validar sin ejecutar

En la UI, botón *Parse* al lado de *Run*. El servidor valida la sintaxis y devuelve el AST si es válida, con la línea y columna del error si no. Útil para hipótesis largas antes de comprometerte a correrlas.

5.7. Guardar hipótesis

Una hipótesis útil se puede convertir en regla Sigma para portarla a otros sistemas (`Hunt \textrightarrow {} Save as Sigma`). También podés guardarla en la librería de hunts compartidos de la sesión para reuso.

Capítulo 6

Modo caza

6.1. Ejecutar un hunt

Desde el panel *Hunt*:

1. Escribir la hipótesis en DSL.
2. Opcional: `min_score` (default 0.0) — descarta paths por debajo de ese score.
3. Opcional: `max_results` (default 500) — corta la salida.
4. *Run*.

La respuesta es una tabla de paths, cada uno con:

- El score compuesto en $[0, 1]$.
- El desglose de los 5 componentes.
- Los nodos del path, en orden.
- La ventana temporal (primer y último evento).

6.2. Paginación

Si hay más resultados que `max_results`, la UI pagina por 50. Podés cambiar el tamaño. Los resultados se persisten en la sesión; podés volver a ellos con *Historial de hunts*.

6.3. Scoring: los 5 componentes

El score compuesto es una combinación ponderada de:

1. **Entity rarity** — qué tan infrecuentes son los nodos individualmente. Valores altos: entidades poco vistas.
2. **Edge rarity** — qué tan infrecuentes son las transiciones. Valores altos: secuencia inusual.
3. **Neighborhood concentration** — qué tan uniforme es el vecindario de cada nodo. Valores altos *penalizan* (centralidad estructural, no anomalía real).
4. **Temporal novelty** — qué tan recientemente aparecieron las entidades. Valores altos: aparecieron al final de la ventana.

5. **GNN threat** — score de modelo GNN si está cargado (por defecto desactivado).

Pesos por defecto: 0.25 / 0.30 / 0.25 / 0.20 / 0.00. Podés ajustarlos desde *Scoring* → *Pesos* o dejarlos en su default.

6.4. Interpretar un path

Un path con score 0.73 y el siguiente desglose:

```
entity_rarity:              0.82
edge_rarity:                0.91
neighborhood_concentration: 0.45
temporal_novelty:           0.64
gnn_threat:                 0.00
```

Se lee: las entidades son raras (0.82), la secuencia es rara (0.91), el vecindario no es particularmente concentrado (0.45, bien), y las entidades aparecieron relativamente tarde (0.64). Score final alto, consistente con actividad sospechosa genuina.

Contraejemplo útil: un path con scores altos en rarity pero *también* alto en `neighborhood_concentration` suele ser una entidad hub (DC, proxy, gateway) que aparece rara simplemente porque conecta muchas cosas. No es anomalía, es topología.

6.5. Workflow de priorización

1. Ordenar por score descendente.
2. Tomar los top-10. Para cada uno:
   - Leer el desglose: ¿qué componente domina?
   - Si `neighborhood_concentration` es muy alto, probablemente es ruido estructural; marcar y seguir.
   - Si `entity_rarity` y `edge_rarity` son altos y `neighborhood_concentration` es bajo, es candidato fuerte. Explorar.
3. Para candidatos fuertes: doble click → modo explorador (cap. 7).
4. Anotar hallazgos.

6.6. Explicar un score

Botón *Explain* sobre un path abre la vista de narrativa: texto humano que describe por qué ese path obtuvo ese score. No reemplaza al desglose numérico — lo acompaña.

6.7. Diferencial entre dos hunts

Hunt → *Diff* permite comparar dos hunts del historial: qué paths aparecieron, cuáles desaparecieron, cuáles cambiaron de score. Útil para:

- Comparar hunts antes y después de ingestar más datos.
- Comparar hipótesis levemente distintas.
- Validar que un cambio de mapping no alteró resultados críticos.

6.8. Recomputar scoring

Si cambias los pesos (o habilitas GNN) sobre resultados ya existentes, usar *Scoring* → *Recompute*. No vuelve a correr el hunt: sólo re-evalúa los paths ya devueltos con los pesos nuevos. Barato.

Capítulo 7

Modo explorador

7.1. Qué es

El modo explorador es la vista nodo-céntrica. Donde el modo caza responde "dada esta forma, ¿qué paths la tienen?", el modo explorador responde "dada esta entidad, ¿a qué está conectada y cuándo?".

Se entra haciendo doble click sobre cualquier nodo de un resultado de hunt, o desde *Buscar*.

7.2. Buscar entidades

En la barra superior: escribir parte del valor (IP, hostname, nombre de usuario) y filtrar opcionalmente por tipo. Resultados aparecen con el conteo de eventos donde participa cada entidad. Click → entra al modo explorador sobre ese nodo.

7.3. Panel de detalle

Para un nodo seleccionado:

- **Propiedades**: tipo, valor, hash si aplica, timestamps de primera y última aparición.
- **Grados**: cuántas aristas entrantes y salientes.
- **Scores precomputados**: entity rarity, neighborhood concentration, temporal novelty (individuales del nodo, antes de ponderar).
- **Tags y notas**: lo que le pusiste.
- **Timeline**: histograma de actividad por hora / día.

7.4. Expandir vecinos

Expand: muestra todos los nodos conectados, con la relación y el timestamp del evento que produjo la arista. Para nodos con muchos vecinos:

Expand grouped: agrupa los vecinos por tipo de relación. Primero ves las relaciones (`Connect` × 1523, `DNS` × 87), después desplegás la que interese. Útil cuando el nodo es un hub.

7.5. Eventos del nodo

Pestaña *Events*: la lista cronológica de eventos donde participa el nodo (como actor o target). Paginada. Cada entrada tiene timestamp, los otros participantes, y atributos del evento.

Para nodos con miles de eventos (hub), la paginación cursor-based evita OOM. Podés filtrar por ventana temporal o por tipo de relación.

7.6. Subgrafo

Selección múltiple: ctrl+click sobre varios nodos, después *Subgraph*. Abre una vista del subgrafo inducido — todas las aristas entre esos nodos. Exportable a GraphSON o JSONL para compartir o diagramar en herramientas externas.

7.7. Cuándo pasar de caza a exploración

- Encontraste un path con score alto y querés entender quién más participa alrededor de esas entidades.
- Un path sospechoso tiene un nodo hub y querés ver de dónde viene la concentración.
- Querés reconstruir narrativa temporal: qué pasó primero, qué vino después.

7.8. Cuándo volver a caza

- Al explorar una IP externa sospechosa, ves que hay otros 3 hosts que también se conectaron allí dentro de 2 horas — escribís un hunt nuevo con esa forma.
- El patrón que descubriste manualmente se puede generalizar a hipótesis reutilizable.

La iteración caza → exploración → caza es el modo normal de trabajo.

Capítulo 8

Notas y tags

8.1. Tags

Una etiqueta es un string corto asociado a una entidad. Tag típicos:

- `sospechoso-c2` — entidad candidata a command-and-control.
- `cuenta-comprometida` — user con evidencia de compromiso.
- `externa-benigna` — IP/domain externo ya triado como no malicioso.
- `hub-corporativo` — entidad hub que hay que ignorar de la caza.

Crear: click derecho sobre un nodo → *Tag* → escribir el tag. Remover: click derecho → *Untag*.

8.2. Listar por tag

Panel → *Tags* muestra todas las etiquetas creadas con sus conteos. Click sobre un tag lista las entidades que lo llevan. Útil para:

- Revisar todos los candidatos de una categoría.
- Armar el inventario de evidencia para un reporte.

8.3. Notas

Una nota es texto libre asociado a una entidad, un path específico, o un hunt. Se crean desde:

- Click derecho sobre un nodo → *Note*.
- Sobre un path en los resultados de hunt → *Annotate*.
- Sobre un hunt en el historial → *Note*.

Soporta Markdown básico. Quedan persistidas en la sesión.

8.4. Workflow de reporte

Flujo típico de construir un reporte desde dentro de la app:

1. Ingerir datos.
2. Correr hunts relevantes; tagear paths sospechosos.
3. Explorar los paths top y tagear entidades clave con `sospechoso-*` o `ioc`.
4. Agregar notas con la narrativa en cada entidad clave ("IP externa con beacon a intervalos regulares, ver captura #12").
5. Al final: *Listar por tag → ioc* da el set de entidades para exportar como IOCs.
6. *Historial de hunts* listado con sus notas da la narrativa completa.

8.5. Interacción con DSL

Una hipótesis puede filtrar por tag:

```
IP {tag("externa-benigna")} -[Connect]-> Host
```

O lo más común: **excluir** entidades ya triadas como no maliciosas para que no contaminen resultados de nuevos hunts.

Los tags viven en la sesión. No se propagan entre sesiones a menos que las exportes explícitamente.

Capítulo 9

Exportación

9.1. IOCs

Archivo → Exportar → IOCs. Parámetros:

- **Filtro por tag** (ej. `ioc`): exporta sólo entidades con ese tag. Default: todas las IPs, domains, hashes marcados en la sesión.
- **Formato**: STIX 2.1 (default), JSON plano, CSV.

El output lista las entidades como indicadores STIX con tipo (`ipv4-addr`, `domain-name`, `file:hashes.SHA256`), primer y último visto, y relaciones explícitas si las agregaste vía tags.

9.2. Publicar IOCs a Sentinel

Si tu organización usa Microsoft Sentinel: *Integraciones → Publicar IOCs*. Requiere:

- `GRAPHHUNTER_IOC_PUBLISH_ENABLED=1` configurada.
- Credenciales Azure configuradas (ver cap. 10).

La app pide confirmación antes de publicar; podés elegir *dry-run* para ver qué se enviaría sin enviar.

9.3. OCSF

Exporta los eventos canonical ya normalizados (ver *Canonical OCSF* en Especificación):

- **Scope**: toda la sesión, o un subset filtrado por ventana temporal, tipos de entidad, o tags.
- **Formato**: JSONL (una línea por evento) o JSON array.

Útil para:

- Reingestar en otra plataforma que hable OCSF.
- Archivo a largo plazo en formato estandarizado.
- Compartir con equipos que usan otras herramientas.

9.4. Subgrafos

Archivo → Exportar → Subgraph a partir de la selección actual (del modo explorador) o de un path específico (del modo caza). Formatos:

- **GraphSON 3.0**: compatible con TinkerPop / Gremlin.
- **JSONL**: lista de vértices y aristas, simple.
- **DOT**: para renderizar con Graphviz.

9.5. Resultados de hunt

Un hunt con sus paths y scores se exporta con *Hunt → Export results*. Formatos CSV o JSON. CSV incluye una columna por componente de score más el compuesto, útil para análisis en planilla.

9.6. Sigma

Una hipótesis útil puede guardarse como regla Sigma (*Hunt → Save as Sigma*). La conversión mapea los nodos y relaciones del DSL a la estructura `detection` de Sigma. No todo DSL es convertible 1:1 — hipótesis que dependen de ventanas temporales o predicados exóticos emiten Sigma con notas sobre qué parte no se tradujo.

9.7. Abrir ticket

Integraciones → Crear ticket. Requiere `GRAPHHUNTER_TICKET_PUBLISH_ENABLED=1` y credenciales de Jira o ServiceNow. El ticket incluye:

- Título y descripción editables.
- Paths seleccionados con su desglose de scoring.
- IOCs tagueados.
- Link a la sesión si está publicada.

Igual que IOCs, soporta *dry-run*.

9.8. Workflow típico de entrega

1. Taguear entidades y paths relevantes durante el análisis.
2. *Exportar IOCs* filtrado por tag — lo mandas al SOC o lo publicas a Sentinel.
3. *Exportar subgrafo* de los paths clave — útil para ilustrar en el reporte.
4. *Crear ticket* con narrativa completa y paths anexados.
5. Mantener la sesión archivada por si se necesita volver.

Capítulo 10

Integraciones

10.1. Azure Sentinel

10.1.1. Ingesta streaming

Integraciones → *Sentinel* → *Conectar*. Te pide:

- `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET` — para un Service Principal con permisos *Reader* en el workspace.
- `SENTINEL_WORKSPACE_ID`.
- Una query KQL (ej. `SigninLogs | where TimeGenerated > ago(24h)`).

Click *Conectar* y la sesión empieza a recibir eventos en vivo. Podés cerrar la conexión con *Desconectar*.

10.1.2. Publish IOCs

Requiere `GRAPHHUNTER_IOC_PUBLISH_ENABLED=1` en el entorno del daemon. Los IOCs se publican como *Threat Intelligence Indicators* en el workspace.

10.1.3. Estado

Integraciones → *Sentinel* → *Status* muestra: conexión activa, eventos recibidos, última query, errores de API.

10.2. MCP (Model Context Protocol)

GraphHunter expone su API como tools MCP. Esto permite que Cursor, Claude Code, Claude Desktop u otros clientes compatibles accedan al grafo vivo desde su asistente.

10.2.1. Setup

En tu cliente MCP (ej. `.cursor/mcp.json`):

```
{
  "mcpServers": {
    "graphhunter": {
      "command": "node",
      "args": ["/path/to/graph-hunter-mcp/dist/src/index.js"],
      "env": {
        "GRAPHHUNTER_API_URL": "http://127.0.0.1:37891",
```

```
        "GRAPHHUNTER_API_TOKEN": "<uuid del app startup>"
      }
    }
  }
}
```

Copiá el UUID que el app imprime al arrancar. Sin token, el MCP se niega a arrancar (salvo que fijes `GRAPHHUNTER_MCP_ALLOW_NO_TOKEN=1`, sólo para dev).

10.2.2. Qué podés pedir

Ejemplos de prompts para el asistente:

- "Buscá todas las IPs externas que se conectaron en la última hora."
- "¿Qué relaciones existen en el grafo entre `alice` y `web-01`?"
- "Corre el hunt `User -[Auth]-> Host -[Connect]-> Host \{k=3\}` y explicame el top 5."

El asistente traduce tu pedido a las tools MCP apropiadas (`search_entities`, `run_hunt`, etc.).

10.2.3. Debug

Si el asistente no ve las tools:

- Verificar que el MCP server arrancó sin errores (`GRAPHHUNTER_MCP_DEBUG=1` para logs verbosos).
- Verificar que el app está corriendo y el token está sincronizado.
- Reiniciar el cliente MCP (a veces hay que bajar y subir Cursor).

10.3. AI backends (LLM)

GraphHunter usa LLM para el carril agéntico (parser_generate, revisión de mappings). Backend configurable:

- **Local GGUF**: setear `GRAPHHUNTER_LOCAL_MODEL_PATH=/ruta/modelo.gguf`. Óptimo para ambientes air-gapped o con datos sensibles.
- **Remoto (OpenAI, Anthropic, Google)**: configurar las credenciales del proveedor (`OPENAI_API_KEY`, etc.) y elegir el backend en *Settings → LLM Backend*.

El LLM nunca ve eventos de producción — sólo muestras explícitas que vos le entregás en el panel agéntico.

10.4. GeoIP (MaxMind)

Para *enrich_ip*: bajar GeoLite2 o GeoIP2 City de MaxMind y apuntar `GRAPHHUNTER_GEOIP_DB=/ruta/GeoLi`
El enriquecimiento es opt-in por llamada; no se auto-enriquece la ingesta.

10.5. Tickets (Jira / ServiceNow)

10.5.1. Jira

```
JIRA_BASE_URL=https://tuorg.atlassian.net
JIRA_EMAIL=tu@email.com
JIRA_API_TOKEN=<token>
JIRA_PROJECT_KEY=SEC
JIRA_ISSUE_TYPE=Task        # opcional, default "Task"
GRAPHHUNTER_TICKET_PUBLISH_ENABLED=1
```

10.5.2. ServiceNow

```
SERVICENOW_INSTANCE=tuorg.service-now.com
SERVICENOW_USERNAME=user
SERVICENOW_PASSWORD=<password>
SERVICENOW_TABLE=incident   # opcional, default "incident"
GRAPHHUNTER_TICKET_PUBLISH_ENABLED=1
```

Cada llamada a *Crear ticket* requiere `confirm:true` o `dry_run:true`. Doble barrera: env var + confirmación.

Capítulo 11

Troubleshooting

11.1. 401 Unauthorized

Síntoma: la UI o el MCP reciben 401.
Causa: token ausente o desalineado.
Fix:

1. En los logs del app buscar la línea `GRAPHHUNTER_API_TOKEN=<uuid>`.
2. Copiar el UUID al cliente (archivo `.env`, config MCP, CLI `--token`).
3. Reiniciar el cliente.

11.2. Connection refused

Síntoma: el cliente no logra conectarse.
Causa: app no corre, puerto distinto, o interfaz de bind incorrecta.
Fix:

- Verificar que el app esté abierto. La API corre sólo mientras el app está vivo.
- En Windows, preferir `127.0.0.1` sobre `localhost` en `GRAPHHUNTER_API_URL`.
- Si cambiaste el puerto con `GRAPHHUNTER_API_PORT`, actualizar el cliente.
- Si configuraste `GRAPHHUNTER_API_HOST=0.0.0.0`, revisar firewall.

11.3. HTML en lugar de JSON

Síntoma: el cliente recibe HTML donde esperaba JSON.
Causa: otro servicio ocupa el puerto 37891 (común si tenés un dev server por ahí).
Fix: parar el servicio conflictivo, o cambiar el puerto del app con `GRAPHHUNTER_API_PORT=NN` y sincronizar el cliente.

11.4. Out of memory con archivos grandes

Síntoma: ingesta falla o el app se congela con EVTX de varios GB.
Fix:

- Activar *Ingesta por disco* (cap. 4).

- Reducir `GRAPHHUNTER_INGEST_BATCH_SIZE` (10000 o 5000).
- Partir el archivo antes e ingestar por partes.

11.5. MCP client no ve tools nuevas

Síntoma: agregaste una tool en el MCP pero el cliente no la lista.
Fix: bajá y subí el cliente MCP (Cursor, Claude Code, etc.). Los clientes cachean la lista de tools al arrancar.

11.6. Datos demo fallan

Síntoma: al ingestar un archivo de `demo_data/`, error de path o de formato.
Fix:

- Verificar que arrancaste el app desde el directorio raíz del proyecto, no desde `apps/tauri/`.
- Si descargaste los datos reales (`DOWNLOAD_REAL_DATA.md`), verificar que estén bajo `demo_data/` con las subcarpetas esperadas.

11.7. GNN scoring siempre 0

Síntoma: el componente `gnn_threat` devuelve 0 para todo path.
Causa: no hay modelo ONNX cargado. Es el comportamiento esperado por default.
Fix si querés activarlo:

- Descargar o entrenar el modelo (ver documentación del core GNN).
- Apuntar `GRAPHHUNTER_GNN_MODEL_PATH` al `.onnx`.
- Aumentar `w5_gnn_threat > 0` en los pesos de scoring.

11.8. Hunt devuelve 0 resultados

Posibles causas:

- Hipótesis demasiado restrictiva (predicados que no matchean nada). Relajar un predicado a la vez y correr de nuevo.
- `min_score` demasiado alto. Bajar a 0.0 para ver todo.
- Ventana temporal (`window`) menor que la distancia real entre eventos. Aumentar.
- Tipos de entidad que no aparecen en el grafo. Verificar en *Catálogo*.

11.9. DLQ creciendo

Síntoma: *Panel → DLQ* acumula eventos.
Causa: mapping mal configurado, evento con campos faltantes, o invariante violado.
Fix:

1. Abrir DLQ, ver la razón de rechazo.
2. Si es un error de mapping, corregir el FieldConfig y reingestar.
3. Si es un evento puntual malformado, reingerir tras corrección o purgar.

11.10. La app está lenta

- Grafo grande (> millones de nodos): usar modo explorador con `Expand grouped` en lugar de expand plano.
- Hunts sin `k` ni `window` sobre grafos grandes toman minutos — acotar.
- Ventana temporal de ingesta muy amplia: considerar sesiones separadas por día / semana.

11.11. Dónde reportar bugs

En `https://github.com/base4sec/graphhunter/issues`. Si abrís un bug, incluir:

- Versión del app (*Ayuda* → *Acerca de*).
- Sistema operativo.
- Los logs del app relevantes (*Ver* → *Logs del sistema*).
- Si aplica, un sample mínimo del archivo que dispara el problema — nunca datos reales sensibles.

Capítulo 12

Preguntas frecuentes

12.1. ¿Reemplaza a mi SIEM?

No. El SIEM es mejor para búsquedas textuales por regla y para alertas en tiempo real. GraphHunter es mejor para investigaciones que requieren razonar sobre estructura (quién-autenticó-dónde- conectó-adónde) sobre ventanas ya delimitadas. Los dos conviven.

12.2. ¿Cuándo me conviene activar scoring?

Casi siempre. Los 5 componentes están calibrados para que un grafo "normal" de oficina produzca distribuciones razonables. Desactivarlo es útil si querés una pasada de exploración pura sin priorización.

12.3. ¿Cuándo conviene activar el GNN threat?

Sólo si tenés un modelo ONNX entrenado para tu entorno. Sin modelo, `gnn_threat` devuelve 0 para todo y subir su peso degrada el score total. Si no sabés, dejalo en 0.

12.4. ¿Puedo tener varias sesiones abiertas?

Sí, en ventanas distintas del app. Cada sesión tiene su propio grafo y su propio workspace. No se comparten tags ni notas entre sesiones.

12.5. ¿El LLM ve mis datos?

Sólo los que vos le entregás explícitamente en el panel agéntico (muestras para proponer mappings). No pasa por él la ingesta productiva. Si usás backend remoto (OpenAI/Anthropic/-Google), esas muestras salen por red; si usás backend local (`GRAPHHUNTER_LOCAL_MODEL_PATH`), quedan en tu máquina.

12.6. ¿Qué pasa si ingesta un log corrupto?

Los eventos que el parser entiende pero violan invariantes van a la DLQ, no al grafo. Los eventos que ni siquiera parsean se loguean como warning y se descartan, con conteo visible en el progreso de ingesta.

12.7. ¿Cómo interpretar un path con gnn_threat = 0?

Significa que el GNN no está activo o no cargó modelo. No es un score de amenaza *baja*, es *no hay señal*. El score compuesto se apoya en los otros 4 componentes.

12.8. ¿Qué pasa si cambio el mapping sobre datos ya ingeridos?

Los eventos ya están en el grafo con el mapping viejo. Cambiar el mapping afecta sólo eventos futuros. Si querés reprocesar, reingerí el archivo fuente.

12.9. ¿Puedo mover una sesión entre máquinas?

Sí. *Archivo → Exportar sesión* guarda un archivo comprimido con el grafo, catálogo, hunts, notas y tags. *Importar sesión* lo restaura en la otra máquina. No se exportan credenciales ni logs del sistema.

12.10. ¿Por qué el Host corporativo aparece con rarity alta?

Probablemente tu ventana de ingesta es corta y ese host no tiene mucha historia en el grafo. El *entity rarity* se calcula sobre el grafo actual, no sobre conocimiento externo. Es una feature, no un bug: te obliga a pensar sobre qué entra al grafo.

12.11. ¿Puedo compartir un hunt con un colega?

- La hipótesis es texto DSL; copiar y pegar alcanza.
- Si querés los mismos resultados: exportar la sesión (archivo).
- Si querés reusar el scoring en una base compartida: publicar el mapping (si es custom) al `mapping_library` común.

12.12. ¿El grafo vive en disco?

Mientras la sesión está abierta, vive en RAM para performance. Al cerrar la sesión (o al exportar explícitamente), se persiste a disco en el directorio de datos del app. Abrir una sesión guardada la carga completa en RAM.

12.13. ¿Qué diferencia hay entre k, window y min_score?

- `k`: simplicidad estructural. Cota de la explosión combinatoria del matcher.
- `window`: separación temporal máxima entre eventos del path.
- `min_score`: umbral *post-hoc* sobre el score compuesto. No afecta el matching, sólo el filtrado de la salida.

12.14. ¿Por qué algunos endpoints requieren una flag de entorno?

Acciones que publican fuera del app (IOCs a Sentinel, tickets a Jira) están detrás de un switch explícito para que nadie las dispare accidentalmente. La flag es la primera barrera; la confirmación por llamada, la segunda.

12.15. ¿Mi hunt tarda mucho. Hay algo que pueda activar?

Sí. El *matcher* acepta varias *flags* de entorno *opt-in* que activan rutas optimizadas incorporadas en la revisión *M-track* de abril 2026. Probar (en orden de seguridad y *lift*):

- `GRAPHHUNTER_LFTJ_PLANNER=1`: si la hipótesis tiene *shape* de K_4 (cuatro entidades en clique con el mismo tipo y mismo tipo de relación entre todas), la dispatcha al enumerador WCOJ que es órdenes de magnitud más rápido.
- `GRAPHHUNTER_YANNAKAKIS=1`: *matcher* de cadenas *output-sensitive* con *semi-join reducer* de dos pasadas. Para cadenas largas con buena selectividad elimina *candidates* muertos antes de enumerar, así que rinde mejor que el DFS cuando la mayoría de los *starts* no van a producir un *match*.
- `GRAPHHUNTER_SEMIJOIN=1`: poda *candidate sets* antes de entrar al DFS sobre cadenas heterogéneas.
- `GRAPHHUNTER_AMAC=1`: *interleaved DFS* con $K = 8$ contextos en vuelo. Útil sobre todo en patrones de ≥ 3 saltos donde la latencia de DRAM domina.
- `GRAPHHUNTER_HUGEPAGES=1` (sólo Linux): reduce *TLB stalls* en grafos grandes.

Todas son seguras: si la *shape* no aplica o la flag no está, se cae al *path* legacy. Detalle completo en el libro *Especificación*, capítulo *Variables de entorno*, y los números medidos en *Paper*, capítulo *Evaluación*.

12.16. ¿Cuándo se activa el compilador IA de *ingest*?

A partir de mayo 2026, los binarios Tauri y CLI vienen con el *compiler LLM-as-compiler plug-and-play*: no hace falta setear `GRAPHHUNTER_LOCAL_MODEL_PATH` ni compilar con *features* extra. La pildora *Ingest:* del *toolbar* muestra el estado real:

- *Ingest: v2 + IA Compiler* — el modelo Phi-3 está *bundled* junto al binario y el *dispatcher* está activo. *Datasets* ambiguos (cero *nodes* canónicos desde la heurística) van automáticamente al *compiler constrained-decode* y la UI muestra un *banner* de confirmación con el *diff* antes de insertar *triples*.
- *Ingest: v2 + LLM (idle)* — el modelo está cargado pero el *dispatcher* fue apagado con `GRAPHHUNTER_LLM_INGEST=0`. La heurística sigue siendo el único *path*.
- *Ingest: v2* — no hay modelo en disco. Sólo heurística. Para activar el *compiler* IA, ejecutar `bash scripts/download-model.sh --stage-bundle` antes del próximo *build*, o setear `GRAPHHUNTER_LOCAL_MODEL_PATH` apuntando a un `.gguf` propio.

El *dispatcher* es seguro por construcción: la salida del LLM se valida contra la gramática *VRL-subset* antes de ofrecerla, y el analista decide aplicar el mapeo IA o quedarse con la heurística. Detalle en el libro *Especificación*, capítulo *Variables de entorno*, entradas `GRAPHHUNTER_LLM_INGEST` y `GRAPHHUNTER_LOCAL_MODEL_PATH`.

12.17. ¿Por qué mi sesión guardada cambió de formato?

A partir de abril 2026, GraphHunter guarda las sesiones en un formato *binario* (*magic bytes* `GHS1`) en lugar del JSON anterior. Es transparente: las sesiones JSON existentes siguen abriendo normalmente (el motor detecta el formato por los primeros 4 bytes del archivo), pero al guardar de nuevo se escriben en binario. Beneficios:

- $\sim 5\times$ más chico en disco.
- $\sim$ 5–10$\times$ más rápido para guardar y abrir.
- *Streaming* en ambos sentidos: la RAM al cargar deja de ser proporcional al tamaño del log y queda acotada por el *buffer* de I/O ($\sim$8 MB). Crítico para sesiones EVTX de 28 GB+ donde el JSON *materializaba* todo el documento en memoria al cargar.

Si necesitás compartir una sesión binaria con una versión vieja del cliente, abrila y volvé a guardarla con un cliente que todavía escriba JSON — ese caso es raro y la migración inversa no está soportada.

12.18. ¿Por qué mi sesión grande abre rápido pero el primer hunt tarda más?

A partir de abril 2026, los *auxiliary indexes* (NLF y k-hop reachability) se construyen *lazy* — al *primer hunt* en lugar de al cargar la sesión. Esto te permite ver el grafo y navegarlo apenas terminó el *streaming load*, en vez de esperar 1–3 minutos a que se construyan los *indexes* antes de mostrarte la UI.

El primer *hunt* sobre una sesión recién abierta paga el costo del *build* ($\sim$30 s para NLF, $\sim$1–2 min para k-hop reachability en un EVTX de 28 GB). *Hunts* subsiguientes son inmediatos — los *indexes* están en memoria (o en *mmap-spilled tempfiles*) y se reutilizan.

Si querés evitar el primer-hunt-lento, dispará un *hunt* trivial apenas cargás la sesión: `User -[Auth]-> User` con *window* amplia es suficiente para forzar el *build* en *background*.

12.19. ¿Cómo escribo una hipótesis con forma de estrella o árbol?

A partir de abril 2026 la *DSL* acepta el separador `;` para introducir cláusulas adicionales que comparten *bindings* con cláusulas anteriores. Una estrella de 3 hojas centrada en `c` se escribe:

```
User c -[Auth]-> Host h1;
c -[Auth]-> Host h2;
c -[Auth]-> Host h3
```

El primer `;` marca la hipótesis como *shape*-`Graph` y el motor dispatcha automáticamente al *tree-walk Yannakakis enumerator*. Las cadenas lineales clásicas (sin `;`) siguen parseándose exactamente como antes y dispatcheando al chain matcher.

Después de un `;`, la cláusula puede arrancar de dos formas:

- Una referencia *bare* a un *binding* declarado antes (como `c` arriba). El motor hereda el *type* del *binding* original.
- Una declaración fresca `Type binding` (e.g., `; User u -[Auth]-> Host h`). Introduce un vértice independiente.

Si el *binding* referenciado no existe en cláusulas anteriores, el *parser* retorna un error que lista los *bindings* válidos para que ajustes el nombre.

12.20. ¿Hay forma de correr varias hipótesis a la vez?

Sí: el endpoint *Tauri* `cmd_run_hunt_batch` acepta una lista de hipótesis y las ejecuta secuencialmente sobre la misma sesión, *pre-warming* los índices auxiliares (NLF y k-hop reachability) una vez antes de la primera consulta. Útil para:

- Baterías de *templates* ATT&CK (decenas de *patterns* sobre el mismo *stream* EDR).
- Comparar varias hipótesis sobre la misma ventana temporal.
- Evitar el *first-hunt-slow* en sesiones recién cargadas: la primera entrada del *batch* paga el *build* de los *indexes* y el resto vuela.

Errores por hipótesis están aislados: una *query* malformada en la posición k falla esa entrada con un `error`, pero el resto del *batch* sigue corriendo. La respuesta incluye un flag `indexesWereWarm` que dice si la *batch* encontró los *indexes* ya construidos (segunda *batch* en adelante) o si los tuvo que construir (primera *batch* tras *open*).

Apéndice A

Glosario

ABI Application Binary Interface. Contrato binario para cruces FFI. *Histórico (pre-2026-05-07)*: GraphHunter cruzaba a `libgraphmatch` (C++) con la versión chequeada al cargar; tras la migración al DFS puro-Rust ya no hay frontera FFI.

Canonical OCSF Formato estándar (OCSF v1.4) al que todo mapping traduce sus eventos. Lingua franca del ingestor.

Drift Desviación estadística observada entre un batch nuevo y el perfil esperado del mapping actual. Dispara re-review.

DSL Domain-Specific Language. El lenguaje de hipótesis de GraphHunter (ver cap. 5).

Entidad Nodo del grafo. Tipada (`User`, `Host`, `IP`, etc.).

FieldConfig Objeto de datos que mapea campos de un log crudo a roles semánticos (`TIME`, `ACTOR`, `TARGET`, ...) y tipos de entidad.

Fast lane Carril determinístico de ingesta productiva. Ejecuta VRL compilado sin involucrar LLM.

FFI Foreign Function Interface. Puente Rust↔ código nativo no-Rust. GraphHunter cruzó FFI a `libgraphmatch` hasta el 2026-05-07; tras esa migración no quedan cruces FFI activos.

GNN Graph Neural Network. Modelo ML que clasifica paths por probable clase de amenaza.

Hipótesis Descripción en DSL de un patrón a buscar en el grafo.

Hunt Una ejecución de hipótesis contra el grafo. Su resultado son paths scoreados.

IOC Indicator of Compromise. Entidad (IP, dominio, hash) marcada como indicador exportable.

k-simplicity Cota estructural sobre la explosión combinatoria del matcher. `\{k=3\}` en una hipótesis.

Learning lane Carril que observa el fast lane y dispara re-review cuando detecta drift.

Mapping library Persistencia de mappings aprobados. Cada mapping tiene `mapping_hash` estable.

Matcher Motor de búsqueda de patrones sobre el grafo. DFS iterativo en Rust (`core/graph-engine/src/graph.r` la intersección sorted-set AVX2 sobrevive como primitiva pura en `simd\_rust.rs` con runtime dispatch.

MCP Model Context Protocol. Protocolo estándar para exponer tools a asistentes de IA.

Model gate Patrón arquitectónico: la funcionalidad ML es opcional; su fallo no degrada el camino base.

ONNX Open Neural Network Exchange. Formato de modelo ML que el GNN usa.

Path Secuencia de nodos conectados por aristas, devuelta por un hunt.

Provenance Metadata que estampa cada evento canonical con el `mapping_hash` que lo produjo. Permite trazabilidad.

Relación Arista del grafo. Tipada (`Auth`, `Connect`, `Spawn`, etc.).

RAII Resource Acquisition Is Initialization. Patrón usado históricamente en los wrappers de `libgraphmatch` (motor removido el 2026-05-07) para gestión automática de memoria a través de `Drop`.

Sesión Workspace aislado con su grafo, catálogo, hunts, tags y notas.

Shearing layers Modelo arquitectónico que agrupa código por velocidad de cambio.

SIMD Single Instruction Multiple Data. Vectorización AVX2 usada por `core/graph-engine/src/simd\_rust.rs` (Rust puro, runtime dispatch) para acelerar la intersección sorted-set en LFTJ.

Slow lane Carril offline donde un LLM propone mappings y un humano los aprueba.

Typestate Patrón de tipado que codifica estados (`Building`, `Finalized`) como parámetros de tipo. Aplicado históricamente a `GmGraph` en `core/matcher-ffi` hasta el 2026-05-07, cuando la frontera FFI se eliminó. El patrón sigue disponible para cualquier máquina de estados futura.

VRL Vector Remap Language. El lenguaje al que se compilan los mappings para ejecución determinística en el fast lane.

Apéndice B

Atajos de teclado

Los atajos asumen layout de teclado estándar. En macOS, reemplazar `Ctrl` por `Cmd`.

B.1. Global

Atajo	Acción
`Ctrl+N`	Nueva sesión
`Ctrl+O`	Abrir sesión
`Ctrl+S`	Guardar sesión
`Ctrl+Shift+S`	Exportar sesión
`Ctrl+W`	Cerrar sesión
`Ctrl+Q`	Salir del app
`Ctrl+,`	Abrir settings
`F1`	Ayuda contextual
`Ctrl+Shift+L`	Ver logs del sistema

B.2. Navegación

Atajo	Acción
`Ctrl+1`	Panel Ingesta
`Ctrl+2`	Panel Catálogo
`Ctrl+3`	Panel Hunt
`Ctrl+4`	Panel Explorador
`Ctrl+5`	Panel Notas/Tags
`Ctrl+6`	Panel DLQ
`Ctrl+F`	Buscar entidades
`Ctrl+G`	Ir a entidad por ID
`Esc`	Cerrar panel modal

B.3. Hunt

Atajo	Acción
`Ctrl+Enter`	Ejecutar hunt
`Ctrl+P`	Parsear sin ejecutar
`Ctrl+D`	Diff con hunt anterior
`Ctrl+E`	Explicar score del path seleccionado
`Ctrl+Shift+E`	Exportar resultados
`Ctrl+H`	Historial de hunts

B.4. Explorador

Atajo	Acción
`Space`	Expandir vecinos del nodo seleccionado
`Shift+Space`	Expand grouped
`T`	Abrir pestaña Timeline
`E`	Abrir pestaña Events
`P`	Abrir pestaña Properties
`S`	Crear subgraph de la selección
`Backspace`	Volver al nodo anterior

B.5. Tags y notas

Atajo	Acción
`Ctrl+T`	Taggear nodo seleccionado
`Ctrl+Shift+T`	Ver entidades por tag
`Ctrl+K`	Crear nota en nodo
`Ctrl+Shift+K`	Listar todas las notas

B.6. Multi-selección

- `Ctrl+Click`: agregar nodo a selección.
- `Shift+Click`: seleccionar rango en listas.
- `Ctrl+A`: seleccionar todo en la vista activa.

Apéndice C

Datos demo

C.1. Qué hay en demo_data/

El repositorio incluye datasets sintéticos listos para probar GraphHunter sin datos reales:

Archivo	Formato	Tamaño aprox.
`fortigate_sample.csv`	FortiAnalyzer CSV	50k eventos
`sysmon_sample.xml`	Sysmon EVTX XML	20k eventos
`sentinel_signin.json`	Sentinel JSON	10k eventos
`cognito_sample.json`	Cognito JSON	5k eventos
`iis_sample.log`	IIS W3C	30k eventos
`csv_generic.csv`	CSV sin mapping	8k eventos

Tabla C.1: Datasets sintéticos embebidos.

Todos los datasets tienen embebidos uno o más *incidentes sintéticos* que los hunts del capítulo 3 deberían detectar. Se usaron para calibrar los componentes de scoring por defecto; podés usarlos como referencia al escribir hipótesis nuevas.

C.2. Incidentes embebidos

C.2.1. fortigate_sample

Lateral movement desde una IP interna a tres hosts en secuencia, con una conexión externa final sospechosa. Hunt recomendado:

```
IP -[Connect]-> IP -[Connect]-> IP {k=3, window=30m}
```

C.2.2. sysmon_sample

Cadena de spawning clásica Office → cmd → powershell. Hunt:

```
Process {name="winword.exe"}
  -[Spawn]-> Process {name="cmd.exe"}
  -[Spawn]-> Process {name="powershell.exe"}
```

C.2.3. sentinel_signin

Impossible travel: el mismo usuario autenticando desde geografías incompatibles en ventana corta. Hunt:

```
User -[Auth]-> Service {window=1h}
```

(Para detectar específicamente impossible travel hace falta inspeccionar los paths manualmente; el enriquecimiento GeoIP ayuda.)

C.2.4. cognito_sample

Intento de password spraying desde IPs distintas contra varios usernames. Hunt:

```
IP -[Auth]-> User {window=10m}
```

C.2.5. iis_sample

Scan de paths sensibles contra servidor IIS. Ventana corta, muchos 404. Hunt:

```
IP -[Connect]-> URL {matches_regex(value, "admin|config|backup")}
{window=5m}
```

C.3. Datos reales

Si querés probar con datasets públicos más grandes, ver `demo_data/DOWNLOAD_REAL_DATA.md`: lista pública de datasets de seguridad (EVTX públicos de labs, exportes OCSF de competencias, etc.) con instrucciones de descarga. Ninguno de esos datos se embebe en el repositorio.

C.4. Reset

Los archivos de `demo_data/` son read-only desde el punto de vista del app. La ingesta los lee sin modificarlos. Para resetear una sesión armada sobre ellos, simplemente cerrar y crear una sesión nueva.